

BUILDING MODERN AUTHENTICATION IN THE BEFORE TIME (2015)

PUTTING THEORY INTO PRACTICES IS HARD

Jeffrey Goldberg

jeffrey@goldmark.org

September 1, 2026

Last revised: September 1, 2026

WHO AM I?

- Jeffrey Goldberg <https://jeffrey.goldmark.org>
- Of the generation of (dropout) academics who ended up as sysadmins.
- Tried (and failed) to solve the password problem in the 1990s.
- Got into information security and cryptography
- 2010–2023: Worked for 1Password, a password manager.
- Not authorized to speak on behalf of 1Password.

OVERVIEW

- A successful password manager from home users.
- Very strong reputation for privacy as well as security and usability.
- Did not host customer data in any form.
- Hooks for users to sync their data using Dropbox and other third party solutions.

WHY RUN A SERVICE?

We were increasingly hit by a number of problems, including:

- Maintaining sync mechanisms with different third party services was getting harder;
- Third party sync services didn't have all security requirements we wanted;
- Users could not reliably share subsets of their data with other members of families or teams.

Running our own service would require, among other things, building an authentication system for it.

REVIEW

For what follows, we need a high level understanding of some of the differences between authentication and encryption as well as to how password cracking (whether for encryption or authentication) is possible.

REVIEW

**PASSWORD MANAGERS ARE NOT BASED ON
AUTHENTICATION**

Authentication involves a prover, P , and a verifier, V . V has the power to grant P access to some resource.

- P has access to a secret, s .
- P proves to V that she knows s , typically sending him s .
- V verifies that s is the correct secret for P
- If the check passes, V grants P access.

- V has the capacity to grant access even if a good proof is not provided.
- V may not be the only entity with the capacity to grant access.

- Alice has a small secret, s , perhaps it is a password.
- Alice has two non-secret mathematical functions E and D .
- $c = E(k, m)$, $m = D(k, c)$, and for all m and k :
 $D(k, E(k, m)) = m$

	Meatspace	Authentication	Decryption
Password	Something you know that is said to a to prove you are authorized to enter	Secret told to system to prove you are authorized to do stuff	Converted to a key necessary to transform data.

REVIEW

CRACKING

Verifiers store something, typically a hash of the password, which if captured by an attacker can be used to for password cracking

Decryption keys are typically derived from password using a key derivation function (KDF) which will including hashing.

CAN HASHES BE REVERSED?

Cryptographic hash functions are (hopefully) pre-image resistant, meaning that given h and a hash function H it is effectively impossible to compute the m of $h = H(m)$.

CAN HASHES BE REVERSED?

Cryptographic hash functions are (hopefully) pre-image resistant, meaning that given h and a hash function H it is effectively impossible to compute the m of $h = H(m)$ if m is chosen uniformly from a large space.

HUMAN CREATED PASSWORDS ARE CRACKABLE

- Humans can't create high entropy passwords;
- Humans can't remember more than a few medium entropy passwords.

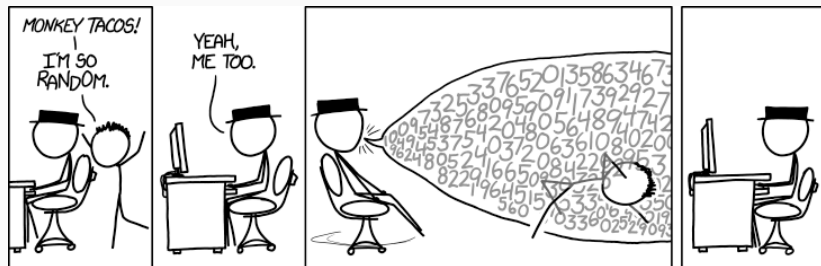


Figure 1: "I'm So Random": [xkcd 1210](#)

SPECIFIC AUTHENTICATION CONCERNS

- Server must not receive decryption secrets during authentication;
- Remain secure even if TLS fails;
- Service does not hold onto anything that is crackable.

In a traditional authentication mechanism, the user password is transmitted to the server, the verifier hashes it, discards the password, and compares the hash to stored hash. Relying on the verification process to discard the secret it receives does not meet the security requirements for a password manager.

Possible solutions

1. Separate password for authentication and decryption
2. Pre-hashing password before transmitting
3. Password-authenticated key exchange (PAKE)

GENERAL AUTHENTICATION DESIDERATA

We not only want the client to prove its identity to the server, but we also want the server to prove its identity to the client.

CI *Proof of client identity*

SI *Proof of server identity*

This is typically handled by TLS, but we wanted the system to retain its security properties if even if TLS fails. Confidence in TLS has (correctly) improved since 2015.

NS *No secrets received*

Traditional password authentication emphatically fails to meet this.

NR *No replay*

An eavesdropper can't learn anything that would allow them to successfully replay an authentication session;

NT *No tampering*

An active attacker who can tamper with the communication can only prevent successful authentication but cannot learn any secrets.

I will refer to the set of these as “NS+”.

KE *Key exchange*

The authentication process results in both sides knowing an encryption key for the session. Proper use of that session key for all communication during the session provides a strong proof that each message comes from a party that successfully went through the authentication process.

There are more slides in this section talking about other desirable properties of an authentication system, but now we will skip ahead to the first slide of the next section, Password-Authenticated Key Exchange (PAKE).

BigH *High entropy*

TOTP is an example of this.

Uniq *Unique long term secrets*

If an attacker obtains the a client secret used for one service, it gives them no information about any other authentication secret. Password reuse fails at this.

NoCrk *Uncrackable server verifiers*

Protects users if server gets breached. Reduces incentives to breach the service.

Use *Reliably and securely usable*

People need to be able to use the system, and the system should be designed to make it easy for people to do things in their own privacy and security interests while hard to do things that go against those interests.

There are features that may be desirable for systems that may need to avoided or created with strong restrictions for a password manager.

Reset *Recoverability*

Any entity that has the power to reset a forgotten or lost authentication secret has the capability to to take over the account.

Alt *Alternative authentication*

Sometimes what is labeled as two-factor authentication is better described as alternative factor authentication. This is extremely dangerous if users are led to believe that it helps keep attackers out.

MF *Multiple factor*

If at least one of multiple authentication factors is not compromised, the authentication process retains its security properties.

PASSWORD-AUTHENTICATED KEY EXCHANGE (PAKE)

A well designed password authenticated key exchange (PAKE) protocol gives us the following security authentication security properties

CI Proof of client identity

SI Proof of server identity

NS No secrets received

NR No replay

NT No tampering

KE Key exchange

1. Generate long term client secret, x , from password, salt (s), etc
2. Compute a mathematically related verifier, v .
3. Send v and s to service. .

AUTHENTICATION: COMPUTE SHARED K

1. Client generates an ephemeral client secret, a .
2. Client generates an ephemeral public key, A , from x and a .
3. Client sends A (along with identity, I) to server.
4. Server generates an ephemeral server secret, b .
5. Server generates an ephemeral public key, B , from v and b .
6. Server sends B to client.
7. Client computes K_c from x , a , and B .
8. Server computes K_a from v , b , and A .

If x and v have the right mathematical relation, then $K_s = K_c$.

AUTHENTICATION: CHECK SHARED K

$K_s = K_c$ only if x and v have the right mathematical relation.

Notation

- “ $\oplus$ ” is exclusive or; “ $|$ ” is reversible concatenation;
- H , an agreed upon cryptographic hash function;
- g , a public parameter used computations;
- s , the salt, shared during enrollment;
- I , the client's identify, set during enrollment;
- K_c and K_s , keys that the client (C) and server (S) have computed.

1. C sends $M_1 = H(H(N) \oplus H(g) \| H(I) | s | A | B | K_c)$ to S.
2. S verifies M_1 .
3. S sends $M_2 = H(A | M_1 | K_s)$ to C.
4. C verifies M_2 .

- Existing libraries we could use;
- Augmented PAKE ($x \neq v$)
- Clear of any patents.

EXISTING LIBRARIES: NOTHING FOR ALL OUR PLATFORMS

Our clients included things written in Objective-C (Mac and iOS), Delphi (Windows), Java (Android), and our planned web-clients that needed to run in Safari, Edge, Firefox, and Chrome.

In principle

A single portable library written in C could be linked to our native clients.

In practice

Our experience doing that with OpenSSL had been unpleasant
More importantly, this wasn't going to work with web-clients.

In principle

In principle web-clients should not have been a thing, not only did they severely limit what sorts of cryptographic libraries we could use, but given the way the client is delivered their security did depend on TLS.

In practice

Web-clients were absolutely necessary, as features (particularly administrative ones) could be developed and deployed quickly to all customers. Adding new UI to native clients was much slower, and releasing an updated native client was subject to additional delays, including app store processes.

Do not roll your own cryptographic implementations!

In principle

We should use elliptic curves for the underlying group.

In practice

The libraries available to us did not expose elliptic curve primitives, but anything with a big integer library could perform the computations for integer fields.

In principle

We should ensure that our implementation defends against timing and other side-channel attacks.

In practice

We did not have access to big integer libraries that did so, and we didn't have the highly specialized skills to do that generally. We did defend against the ones that we easily could.

LEAKS LIKE A SIEVE



Figure 2: Leeks like a sieve

SECURE REMOTE PASSWORD (SRP 6A)

In principle

Use a PAKE with cleaner and clearer security proofs and analysis than SPR-6a

In practice

SRP-6a was only viable option, given that we had to implement it ourselves and was free of any potential patent claims.

A ONCE FRESH MEME

Do not use SPR for any new projects today. It was a necessary choice for 1Password in 2015; there are far better PAKEs available now.



Figure 3: Something I tweeted a year after launch

LESSONS FROM ROLLING OUR OWN

Pointing even very talented developers at a standard and expecting them to implement well if they don't understand why the protocol works leads to problems

- Without an understanding of how the pieces fit together, code can be wildly misfactored.
- Developers didn't know which parts of computation where cryptographic secrets.
- Allowed a seemingly harmless shortcut in not padding a number before hashing that lead to a Heisenbug years later.

In the years that followed, a common core library for all clients was developed in Rust. That was an intensive, multi-year project. Additionally greater cryptographic expertise was brought on board.